

Apprentissage statistiques pour l'IA

BUT SD 3A S6
2023-2024

Partie 1 : Rappels

Christophe REYPE



© 2023 Christophe REYPE

Document sous licence Creative Commons Attribution – Partage dans les mêmes conditions 4.0 International (CC BY-SA 4.0)

<https://creativecommons.org/licenses/by-sa/4.0/>

Table des matières

Introduction

Les données

La régression

La classification

Limites des méthodes de modélisation "classiques"

Problématique

Objectif général : Comprendre ou prédire le comportement des données.

Objectifs du cours :

- ▶ Comprendre ou prédire le comportement des données,
- ▶ Comprendre les limites des méthodes de modélisation,
- ▶ En pratique, être capable de choisir une méthode de modélisation et d'interpréter ses résultats.

Régression et Classification

Régression :

- ▶ Déterminer une loi à partir des données,
- ▶ Associer à un individu une valeur réelle,
- ▶ Ex : estimer la taille d'un individu, prédire le coût d'une décision...

Classification :

- ▶ Déterminer une manière de ranger les données,
- ▶ Associer à un individu une catégorie,
- ▶ Ex : reconnaître un animal sur une photo, prédire dans quelle catégorie sera un individu...

A faire :

- ▶ Vérifier que le modèle s'applique sur les données,
- ▶ Connaître les limites du modèle,
- ▶ Interpréter les résultats EN TENANT COMPTE des limites du modèle.

Table des matières

Introduction

Les données

La régression

La classification

Limites des méthodes de modélisation "classiques"

Notations

- ▶ Effectif : n individus/observations notés $\mathbf{x} = (x_1, \dots, x_n)$.
- ▶ Observation : l'individu i a pour variables explicatives $x_i = (x_i^{\{1\}}, \dots, x_i^{\{d\}})$ avec d le nombre de variables d'intérêts
- ▶ La variable cible pour l'individu i est
 - Régression : $y_i \in \mathbb{R}$,
 - Classification : $y_i \in \{1, \dots, C\}$ avec C le nombre de catégories.
- ▶ L'estimation de la variable cible est obtenu par $\hat{y}_i = f(x_i)$.
- ▶ Il faut minimiser l'erreur du modèle : $\sum_{i=1}^n \|y_i - \hat{y}_i\|$

Exemple

Identifiant	Date de naissance	Musique préférée	Taille	...
0001	10/03/2002	Never Gonna Give You Up	178	...
0002	15/08/2003	Sandstorm	163	...
...	...	...	...	...

Table – Exemple de tableau de données.

Questions :

- ▶ Peut-on prédire la distance entre le logement de l'individu et l'IUT ?
- ▶ Peut-on prédire la couleur préférée d'un individu ?

Table des matières

Introduction

Les données

La régression

La classification

Limites des méthodes de modélisation "classiques"

Présentation de la régression

Définition : Prédire une variable quantitative.

Exemples de problématiques :

- ▶ Prédire la superficie qui risque d'être brûlée lors d'un incendie,
- ▶ Prédire les dépenses d'un individu,
- ▶ Prédire la production d'une exploitation agricole.

Visualisation de la régression

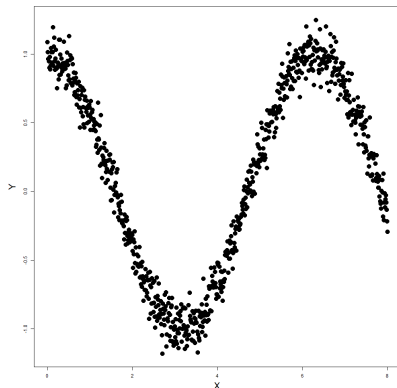


Figure – Exemple de données pour la régression avec X la variable explicative et Y la variable à expliquer.

Exemples de modèle de régression classique

- ▶ régression linéaire,
- ▶ régression polynomiale,
- ▶ régression Ridge/ Lasso,
- ▶ modèle linéaire généralisé (GLM),
- ▶ ...

Table des matières

Introduction

Les données

La régression

La classification

Limites des méthodes de modélisation "classiques"

Présentation de la classification

Définition : Prédire une variable qualitative.

Exemples de problématiques :

- ▶ Prédire les zones à risque d'incendie,
- ▶ Prédire si un individu est susceptible de rejoindre un programme de fidélité,
- ▶ Prédire si une exploitation agricole va être touchée par une épidémie.

Visualisation de la classification

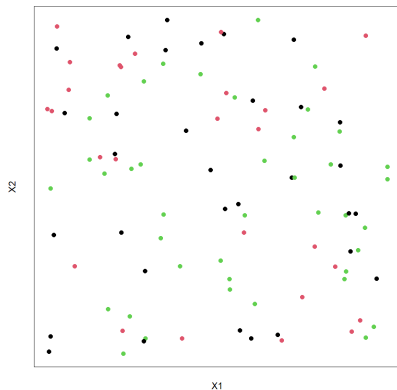


Figure – Exemple de données pour la classification avec $X = (X_1, X_2)$ la variable explicative et la couleur Y la variable à expliquer.

Exemples de modèle de classification classique

- ▶ k -means
- ▶ k-Nearest Neighbors
- ▶ arbre de classification...

Introduction

L'objectif de l'étude est de développer une méthode de classification des données.

L'idée de la classification est de séparer les données selon des classes. Ces classes sont définies de sorte à minimiser les différences entre les individus d'une même classe et de maximiser les différences entre les individus de classes différentes.

- ▶ k -moyennes (k -means)
- ▶ classification hiérarchique (hierarchical clustering)

Table des matières

Introduction

Les données

La régression

La classification

Limites des méthodes de modélisation "classiques"

Limites des méthodes de modélisation "classiques" :

- ▶ Jeux de données avec beaucoup d'individus et/ ou beaucoup de variables,
- ▶ Repose sur des hypothèses probabilistes,
- ▶ Modèle construit à partir de la réalité et pas nécessairement le mieux ajusté.

Apprentissage statistiques pour l'IA

BUT SD 3A S6
2023-2024

Partie 1 : Agrégation de modèle

Christophe REYPE



© 2023 Christophe REYPE

Document sous licence Creative Commons Attribution – Partage dans les mêmes conditions 4.0 International (CC BY-SA 4.0)

<https://creativecommons.org/licenses/by-sa/4.0/>

Table des matières

Agrégation de modèles : méthodes ensemblistes

Bootstrap

Bagging

Le bagging d'arbres de décision : les forêts aléatoires ou random forest

Boosting

Récapitulatif bagging, forêts aléatoires et boosting

Introduction

Objectif : Développer une méthode de classification des données.

Problématique :

- ▶ Peut-on augmenter la performance d'une méthode de classification ?
- ▶ Peut-on diminuer la variabilité d'une méthode de classification ?

Présentation de l'agrégation de modèle

Principe :

1. Construire plusieurs classifieurs,
2. Pour chaque données, prédire la classe obtenue par chaque classifieur,
3. Pour chaque données, lui associer la classe majoritairement obtenue.

Les mains dans le cambouis

Présentation du problème : On veut construire un classifieur qui range les données dans deux classes différentes. Pour cela, on utilise N classifieurs indépendants chacun ayant une probabilité de donner la mauvaise classe p_i , avec i le numéro du classifieur.

Questions :

- ▶ Pour $N = 4$,
 - ▶ Présentez tous les cas possibles.
 - ▶ Quelle est la probabilité que le modèle agrégé prédise la bonne classe ?
- ▶ En posant pour tous les i $p_i = p$,
 - ▶ Quelle est la probabilité que k classifieurs prédisent la mauvaise classe ?
 - ▶ Quelle est la probabilité que la majorité des classifieurs prédisent la mauvaise classe ?

Table des matières

Agrégation de modèles : méthodes ensemblistes

Bootstrap

Bagging

Le bagging d'arbres de décision : les forêts aléatoires ou random forest

Boosting

Récapitulatif bagging, forêts aléatoires et boosting

Bootstrap

Intérêt Créer des échantillons à partir des données initiales.

Echantillon bootstrap Echantillon construit par un tirage aléatoire avec remise des données initiales.

Remarques :

- ▶ Une même observation peut apparaître dans plusieurs échantillon et même plusieurs fois dans un même échantillon.
- ▶ Chaque classifieur est entraîné sur un échantillon bootstrap différent.
- ▶ Le tirage avec remise devrait assurer l'indépendance des classifieurs.

Table des matières

Agrégation de modèles : méthodes ensemblistes

Bootstrap

Bagging

Le bagging d'arbres de décision : les forêts aléatoires ou random forest

Boosting

Récapitulatif bagging, forêts aléatoires et boosting

Bagging

Principe : Le Bootstrap AGGREGatING ou bagging consiste en la création d'un classifieur fort à partir de classifieurs faibles entraînés sur des échantillons bootstraps.

Propriétés :

- ▶ Le bagging a le même biais que les classifieurs faibles mais à une plus faible variance.
- ▶ Il faut que les classifieurs faibles soient le moins corrélés entre eux possible.
- ▶ Les classifieurs faibles doivent avoir un biais le plus petit possible (donc une forte variance). Les arbres de décisions ont cette propriété.

Algorithme bagging

Classification

1. Construire les N classifieurs :
 - ▶ Tirer un échantillon bootstrap,
 - ▶ Estimer le classifieur sur l'échantillon bootstrap.
2. Pour chaque observation, renvoyer la classe majoritairement obtenues par les classifieurs.

Régression

1. Construire les N régressions :
 - ▶ Tirer un échantillon bootstrap,
 - ▶ Estimer la régression sur l'échantillon bootstrap.
2. Pour chaque observation, renvoyer la moyenne obtenue par les régressions.

Schéma explicatif du bagging

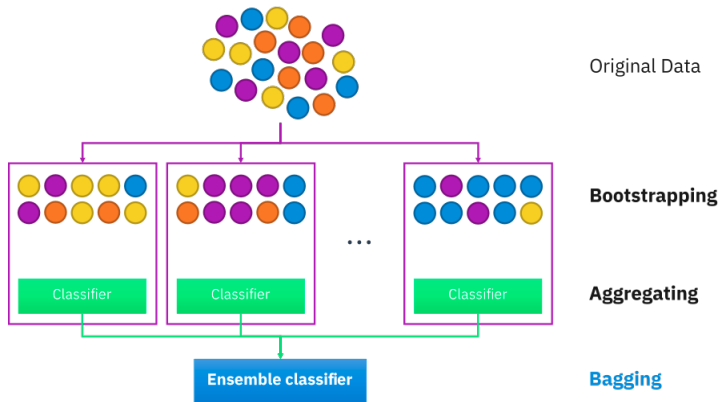


Figure – Sirakorn, CC BY-SA 4.0

<https://creativecommons.org/licenses/by-sa/4.0>, via Wikimedia Commons

Table des matières

Agrégation de modèles : méthodes ensemblistes

Bootstrap

Bagging

Le bagging d'arbres de décision : les forêts aléatoires ou random forest

Boosting

Récapitulatif bagging, forêts aléatoires et boosting

Arbre de décision

Principe Classifier des données selon des décisions successives sur des caractéristiques.

Propriétés :

- ▶ La classification se fait par une succession de décisions élémentaires.
- ▶ Les décisions fournies sont facilement interprétables (boîte blanche).
- ▶ Il s'agit d'un algorithme glouton (greedy algorithm) : un choix optimum local est effectué étape par étape pour obtenir un optimum global.
- ▶ Un arbre de profondeur (nombre de décisions successives) 1 est appelé une souche de classification (decision stump)

Schéma explicatif de l'arbre de décision

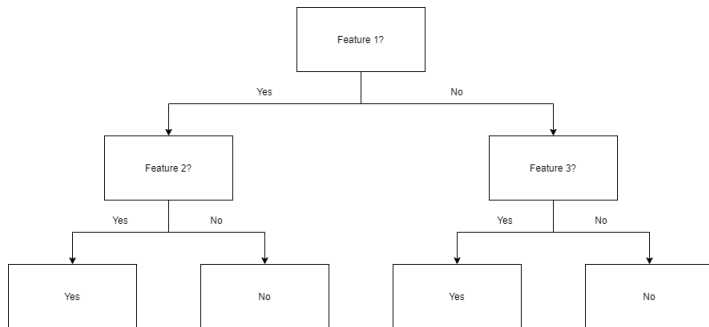


Figure – CollaborativeGeneticist, CC BY-SA 4.0

<https://creativecommons.org/licenses/by-sa/4.0>, via Wikimedia Commons

Algorithme CART

Classification And Regression Trees

1. Pour chaque variable u , déterminer le seuil C_u qui sépare le mieux les observation,
2. Sélectionner la variable et son seuil qui minimisent le critère de segmentation,
3. Séparer les observations en deux sous-ensembles selon le seuil,
4. Pour ces sous-ensembles recommencer à l'étape 1 si la condition d'arrêt n'est pas satisfaite.

Critère de segmentation :

- ▶ Classification : critère de Gini $C = \sum_{k=1}^K p_k(1 - p_k)$ avec p_k la proportion d'observations appartenant à la classe k ,
- ▶ Régression : erreur quadratique moyenne (MSE) $\frac{1}{n} \sum (y_i - \hat{y})^2$.

Condition d'arrêt : Le sous-ensemble ne contient des observations que d'une classe OU l'arbre est trop profond.

Caractéristiques de l'algorithme CART

Propriétés de l'algorithme CART :

- ▶ Toutes les décisions sont interprétables.
- ▶ Les décisions étant sélectionnées parmi les valeurs possibles des x^u , l'algorithme peut être sous optimal (exemple : dans $\mathbb{R}^2$, deux classes séparé par une droite).
- ▶ Pour éviter l'overfitting, les décisions impliquant un trop faible nombre d'observations sont supprimées.

Forêt aléatoire

Principe :

1. Construire N arbres de décision :
 - ▶ Tirer aléatoirement d variables,
 - ▶ Construire un échantillon bootstrap en considérant ces d variables,
 - ▶ Construire un arbre de décision sur l'échantillon bootstrap
Tirer un échantillon bootstrap,
 - ▶ Estimer le classifieur sur l'échantillon bootstrap.
2. Pour chaque observation, renvoyer la classe majoritairement obtenues par les classifieurs.

Caractéristiques :

- ▶ Les forêts aléatoires sont des boîtes noires.
- ▶ Par défaut dans la fonction `randomForest` du package R du même nom, 500 arbres de décisions sont créés chacun sélectionnant $m = \sqrt{d}$ variables pour le critère de sélection.

Table des matières

Agrégation de modèles : méthodes ensemblistes

Bootstrap

Bagging

Le bagging d'arbres de décision : les forêts aléatoires ou random forest

Boosting

Récapitulatif bagging, forêts aléatoires et boosting

Boosting

Principe :

1. Choisir un classifieur,
2. Donner un poids plus important aux observations mal classées et un poids plus faible aux observations bien classées,
3. Pour un nombre d'itération N ,
 - ▶ Choisir le classifieur qui classe le mieux les observations mal classées,
 - ▶ Donner un poids plus important aux observations mal classées et un poids plus faible aux observations bien classées.

Caractéristiques :

- ▶ A chaque étape, le jeu de données est séparé en deux groupes : les données correctement classées et les autres.
- ▶ A chaque étape une pondération est affectée à chaque groupe : faible pour le groupe correcte et forte pour le groupe incorrecte.
- ▶ Le classifieur choisi pour l'étape suivante est celui qui classe le mieux dans le groupe mal classé.

Schéma explicatif du boosting

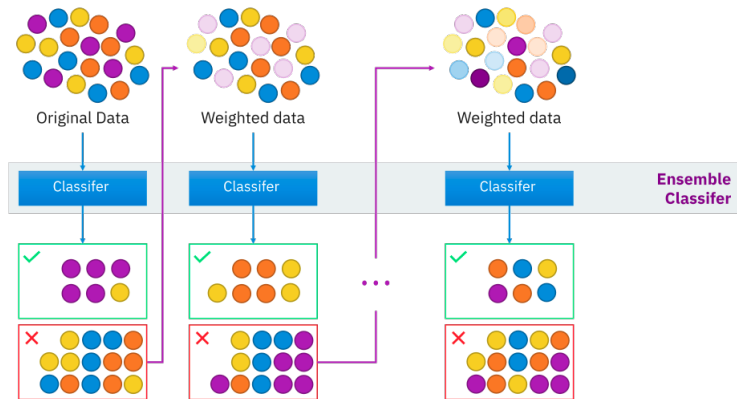


Figure – Sirakorn, CC BY-SA 4.0

<https://creativecommons.org/licenses/by-sa/4.0>, via Wikimedia Commons

Adaboost

Principe : L'ADaptive BOOSTing sélectionne les classifieurs faibles en minimisant le risque empirique. Le classifieur fort prédit la classe par un vote pondéré des classes prédites. Un classifieur performant aura un poids important.

Caractéristiques :

- ▶ Les observations sont stimulées (boosted) par le choix de la pondération.
- ▶ Les arbres de décision sont souvent utilisés comme classifieur. Le critère de segmentation est le coût des erreurs.
- ▶ Les souches de classification sont souvent utilisés comme classifieur.
- ▶ A chaque étape, les pondérations sont normalisées.
- ▶ Un nombre trop grand d'étape conduit à un overfitting.

Construction de l'Adaboost : Notations

- ▶ Une observation est notée x_i sa classe réelle est notée y_i .
- ▶ Un classifieur est noté h , la classe prédite de x_i est notée $h(x_i)$. Le nombre de classifieurs est M .
- ▶ Le poids des x_i est ω_i et de chaque classifieur h_m est α_m .
- ▶ Le classifieur fort est $K_M = \sum_{m=0}^{\{m\}} \alpha_m h_m$. La classe prédite par le classifieur fort pour x_i est :
$$K_M(x_i) = \arg \max_k \sum_{m=1}^{\{m\}} \alpha_m \mathbb{1}_{\{h_m(x_i)=k\}}.$$
- ▶ Pour une fonction de perte ℓ et un classifieur fort K_M , le risque empirique est :

$$E = \sum_{i=1}^n \ell(y_i, K_M(x_i)).$$

- ▶ La fonction de perte de l'Adaboost est la perte exponentielle ($\ell(y_i, K_M(x_i)) = \exp(-1_{\{K_M(x_i)=k\}})$). Pour diminuer l'intensité du mécanisme adaptatif, il est possible de tenir compte du taux d'apprentissage $\lambda \in [0, 1]$:
$$\ell(y_i, K_M(x_i)) = \exp(-\lambda 1_{\{K_M(x_i)=k\}}).$$

Algorithme Adaboost

Algorithme Adaboost Poser M , et initialiser les poids $\omega_i^{\{0\}}$ uniformément.

► Pour $m = 0, \dots, M$

1. Sélectionner le classifieur h_m qui minimise le coût des erreurs :

$$\epsilon_m = \sum_1^n \omega_i^{\{m\}} 1_{\{h_m(x_i)=k\}} / \sum_1^n \omega_i^{\{m\}}.$$

2. Calculer le poids de ce classifieur : $\alpha_m = \log \left(\frac{1-\epsilon_m}{\epsilon_m} \right)$.

3. Mettre à jour les poids des observations :

$$\omega_i^{m+1} = \omega_i^{\{m\}} \exp(\alpha_m 1_{\{h_m(x_i)=k\}})$$

4. Normaliser les poids des observations.

Table des matières

Agrégation de modèles : méthodes ensemblistes

Bootstrap

Bagging

Le bagging d'arbres de décision : les forêts aléatoires ou random forest

Boosting

Récapitulatif bagging, forêts aléatoires et boosting

Récapitulatif du bagging

Caractéristiques

- ▶ A chaque itération, l'apprentissage se fait sur un échantillon bootstrap différent et sur l'ensemble des prédicteurs.
- ▶ A chaque itération, le modèle est performant sur l'ensemble des observations.
- ▶ Les modèles ont le même poids.

Avantages et inconvénients

- ▶ Réduction de la variance par la moyennisation des modèles.
- ▶ Perte d'interprétabilité des arbres de décisions.
- ▶ Peu efficace sur les souches de décision.
- ▶ Possibilité de paralléliser.
- ▶ Convergence rapide.
- ▶ Pas de sur-apprentissage.
- ▶ Simple mais moins discriminant.

Récapitulatif des forêts aléatoires

Caractéristiques

- ▶ A chaque itération, l'apprentissage se fait sur un échantillon bootstrap différent et sur un sous-ensemble des prédicteurs.
- ▶ A chaque itération, le modèle est performant sur l'ensemble des observations, mais moins que pour le bagging du fait du sous-ensemble de prédicteurs considérés.
- ▶ Les modèles ont le même poids.

Avantages et inconvénients

- ▶ Grande réduction de la variance par la moyennisation des modèles.
- ▶ Perte d'interprétabilité des arbres de décisions.
- ▶ Efficace sur les souches de décision.
- ▶ Possibilité de paralléliser.
- ▶ Convergence rapide.
- ▶ Pas de sur-apprentissage.
- ▶ Plus efficace que le bagging et que le boosting (sauf si les prédicteurs discriminants sont rares).

Récapitulatif du boosting

Caractéristiques

- ▶ A chaque itération, l'apprentissage se fait sur la totalité des données et sur l'ensemble des prédicteurs.
- ▶ A chaque itération, le modèle est performant sur un sous-ensemble des observations (les observations mal classées à l'itération précédente).
- ▶ Les modèles ont un poids différent.

Avantages et inconvénients

- ▶ Peut diminuer le biais et la variance mais peut aussi augmenter la variance.
- ▶ Perte d'interprétabilité des arbres de décisions.
- ▶ Très efficace sur les souches de décision.
- ▶ Pas naturellement parallélisable.
- ▶ Convergence lente.
- ▶ Risque de sur-apprentissage si trop d'itérations.
- ▶ Plus efficace que le bagging.

Apprentissage statistiques pour l'IA

BUT SD 3A S6
2023-2024

Partie 2 : Machines à vecteurs supports

Christophe REYPE



© 2023 Christophe REYPE

Document sous licence Creative Commons Attribution – Partage dans les mêmes conditions 4.0 International (CC BY-SA 4.0)

<https://creativecommons.org/licenses/by-sa/4.0/>

Table des matières

Données linéairement séparables

Classification et problème d'optimisation

Les marges souples

Données non linéairement séparables

Conclusion sur les classifieurs

Régression à vecteurs supports

Pour aller plus loin : Reformulation en problème d'optimisation

Définition de données linéairement séparables

Définition : Les données sont linéairement séparables si ces classes peuvent être séparées par des hyperplans.

Hyperplan : Un hyperplan d'un espace vectoriel E en K -dimensions est un sous-espace vectoriel de E de dimension $K - 1$.

Exemples :

- ▶ En 2D, un hyperplan est une droite,
- ▶ En 3D, un hyperplan est un plan.

Visualisation de données linéairement séparables

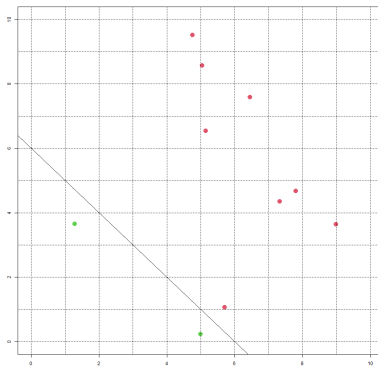


Figure – Exemple de données linéairement séparables en 2D.

En pratique

Important : Dans la suite, nous ne considérons que le cas de deux classes.

Notations :

- ▶ Les observations sont notées x_i leur label y_i avec $y_i \in \{-1, 1\}$.
- ▶ Les hyperplans séparateurs de paramètres (ω, b) vérifient pour tout i

$$\begin{cases} \omega \cdot x_i + b > 0 & \text{si } y_i = 1 \\ \omega \cdot x_i + b < 0 & \text{si } y_i = -1 \end{cases}$$

- ▶ $x_i \cdot x_j = \sum_{u=1}^d x_i^{\{u\}} x_j^{\{u\}}$ avec pour tout i :
 $x_i = (x_i^{\{1\}}, \dots, x_i^{\{d\}})$.

Les mains dans le cambouis

- ▶ Définissez les paramètres (ω, b) de l'hyperplan séparateur dans les cas 2D (voir figure visualisation de données).
- ▶ Donnez les paramètres d'un hyperplan dans le cas de données en 4D.
- ▶ Comment choisir l'hyperplan séparateur ?

Choix de l'hyperplan séparateur

- ▶ La plus petite distance entre les observations et l'hyperplan séparateur est $\frac{1}{\|\omega\|_2} \min_i \{y_i(\omega \cdot x_i + b)\}$.
- ▶ La marge est le double de la plus petite distance entre les observations et l'hyperplan séparateur.
- ▶ Les paramètres de l'hyperplan à plus grandes marges sont $\arg \max_{\omega, b} \left\{ \frac{1}{\|\omega\|_2} \min_i \{y_i(\omega \cdot x_i + b)\} \right\}$.
- ▶ Les observations se situant sur la marge sont appelées vecteurs supports.

Marge et vecteur support

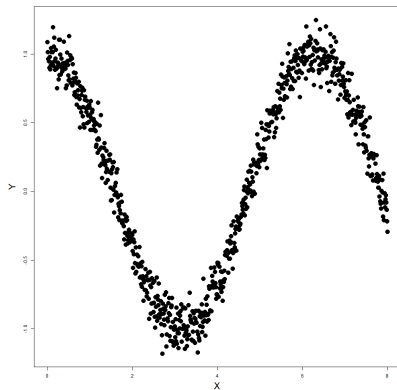


Figure – Exemple de données linéairement séparables en 2D.

Machines à vecteurs supports

Principe : Les machines à vecteur supports ou SVM (Support Vector Machines) est une technique de classification de données.

Cas de données linéairement séparables : L'hyperplan séparateur est celui avec les plus grandes marges.

Remarque : Les SVM n'utilisent que les vecteurs supports pour classifier les données

Table des matières

Données linéairement séparables

Classification et problème d'optimisation

Les marges souples

Données non linéairement séparables

Conclusion sur les classifieurs

Régression à vecteurs supports

Pour aller plus loin : Reformulation en problème d'optimisation

Classification et problème d'optimisation

Reformulation

- ▶ Sans perte de généralité, les vecteurs supports vérifient : $y_i(\omega \cdot x_i + b) = 1$ (car la distance est invariante par multiplication de ω et b par un scalaire non nul).
- ▶ La recherche de (ω, b) revient au problème d'optimisation appelé problème primal

$$\begin{cases} \arg \min_{\omega, b} \frac{1}{2} \|\omega\|_2^2 \\ y_i(\omega \cdot x_i + b) \geq 1 \quad \forall 1 \leq i \leq n \end{cases}$$

- ▶ Ce problème d'optimisation est convexe donc admet une unique solution.

Table des matières

Données linéairement séparables

Classification et problème d'optimisation

Les marges souples

Données non linéairement séparables

Conclusion sur les classifieurs

Régression à vecteurs supports

Pour aller plus loin : Reformulation en problème d'optimisation

Les marges souples

Idee : Comment intégrer le bruit dans les observations ?

Intérêts des marges souples : (soft margin)

- ▶ Permet de tenir compte des perturbations dans les données qui rendent les classes non linéairement séparables.
- ▶ Permet des erreurs de classification et ainsi éviter l'overfitting.

Variable d'écart à la marge (slack variable)

- ▶ Pour l'observation x_i

$$\xi_i = \begin{cases} 0 & \text{si } y_i(\omega \cdot x_i + b) \geq 1 \\ |y_i - (\omega \cdot x_i + b)| & \text{sinon} \end{cases}$$

- ▶ $\xi_i = \max\{0, 1 - y_i(\omega \cdot x_i + b)\}$

Problème primal de la SVM à marge souple

Problème primal Soit l'hyperparamètre $C > 0$

$$\begin{cases} \arg \min_{\omega, b} \frac{1}{2} \|\omega\|_2^2 + C \sum_{i=1}^n \xi_i \\ \xi_i \geq 1 - y_i(\omega \cdot x_i + b) \quad \forall 1 \leq i \leq n \\ \xi_i \geq 0 \quad \forall 1 \leq i \leq n \end{cases}$$

Rôle de C

- ▶ Si C est grand : forte pénalisation des ξ_i donc la marge souple est presque une marge dure.
- ▶ Si C est proche de 0 : les ξ_i peuvent être grand donc risque d'underfitting.

Table des matières

Données linéairement séparables

Classification et problème d'optimisation

Les marges souples

Données non linéairement séparables

Conclusion sur les classifieurs

Régression à vecteurs supports

Pour aller plus loin : Reformulation en problème d'optimisation

Passage en grandes dimensions

- ▶ Dans un espace de K dimensions, la classification de $K + 1$ observations dans deux classes est un problème de classification linéairement séparable.
- ▶ Plonger les observations dans un espace de dimensions plus grandes appelé espace de redescription (feature space) permet de transformer le problème en classification linéairement séparable.
- ▶ Les observations sont plongées dans l'espace de redescription au moyen d'une fonction de plongement ϕ .
- ▶ En pratique il n'est pas nécessaire de déterminer explicitement ϕ , il suffit d'avoir les $\phi(x_i) \cdot \phi(x_j)$.

Astuce du noyau

- Le classifieur s'écrit pour une nouvelle observation x

$$f(x) = \omega \cdot \phi(x) + b = \sum_{i=1}^n \lambda_i y_i \phi(x_i) \cdot \phi(x) + b \quad (1)$$

- La fonction $k(x_i, x) = \phi(x_i) \cdot \phi(x)$ est appelée un noyau.
- Le classifieur s'écrit pour une nouvelle observation x

$$f(x) = \sum_{i=1}^n \lambda_i y_i k(x_i, x) + b \quad (2)$$

- Le calcul de f par le noyau est plus rapide que par le calcul explicite de ω

Ingénierie des noyaux

Définition Un noyau est une fonction k définie sur $\mathbb{R}^d \times \mathbb{R}^d$ à valeurs dans $\mathbb{R}$ continue, symétrique et telle que pour tout $x_1, \dots, x_n$ de $\mathbb{R}^d$ la matrice symétrique $\left(k(x_i, x_j) \right)_{1 \leq i, j \leq n}$ est positive.

Exemple de noyaux

- ▶ Noyau linéaire : $k(x_i, x_j) = x_i \cdot x_j$
- ▶ Noyau polynomial : $k(x_i, x_j) = (x_i \cdot x_j + \rho)^\delta$ où $\rho > 0$ et δ un entier strictement positif.
- ▶ Noyau RBF (Radial basis function) ou noyau gaussien :
 $k(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|_2^2)$ où $\gamma > 0$.

Remarque : Les noyaux peuvent être définis sur des chaînes de caractères, des ensembles ou mêmes des graphes. Par exemple pour les protéines, les noyaux sont définis sur les 22 acides aminés.

Construction de noyaux valide

Soient k_1 et k_2 des noyaux valides, $c > 0$, A une matrice symétrique positive, $x = (x_a, x_b)$ avec x_a définie sur l'espace E_a et x_b sur l'espace E_b ainsi que k_a et k_b des noyaux définis sur ces espaces

- ▶ $k(x, y) = ck_1(x, y) + k_2(x, y)$
- ▶ $k(x, y) = x^T A y$
- ▶ $k(x, y) = k_1(x, y)k_2(x, y)$
- ▶ $k(x, y) = k_a(x_a, y_a) + k_b(x_b, y_b)$
- ▶ $k(x, y) = k_a(x_a, y_a)k_b(x_b, y_b)$

Généralisation des SVM à K classes

One-versus-all K classifieurs binaires sont construits. Donnez aux données le label 1 pour une classe et -1 pour les autres classes. Le classifieur avec la valeur de confiance (la marge) la plus élevée donne la bonne classe.

One-versus-one $K(K - 1)/2$ classifieurs binaires pour confronter chaque paire de classes. La classe sera celle la plus attribuée à x_i .

Table des matières

Données linéairement séparables

Classification et problème d'optimisation

Les marges souples

Données non linéairement séparables

Conclusion sur les classifieurs

Régression à vecteurs supports

Pour aller plus loin : Reformulation en problème d'optimisation

Conclusion sur les classifieurs

Problème en grande dimensions :

- ▶ Si le nombre de dimension est grand devant le nombre d'observations alors la classification est linéairement séparable $\Rightarrow$ utiliser une SVM classique.
- ▶ Il est important de se poser la question de la pertinence de travailler dans un espace de dimensions aussi élevées.

Classification non linéairement séparable :

- ▶ Dans ce cas il faut préférer une SVM à noyau (par exemple le noyau RBF) ou l'Adaboost avec des souches de décisions.

En pratique : Il est impossible de savoir à l'avance si le problème est linéairement séparable ou non.

Table des matières

Données linéairement séparables

Classification et problème d'optimisation

Les marges souples

Données non linéairement séparables

Conclusion sur les classifieurs

Régression à vecteurs supports

Pour aller plus loin : Reformulation en problème d'optimisation

Du SVM au SVR

Notations :

- ▶ Les observations sont de la forme (x_i, y_i) avec x_i un vecteur de variables explicatives et y_i une variable expliquée réelle.

Support Vector Regression

- ▶ Le prédicteur est de la forme $f(x_i) = \omega \cdot x_i + b$.
- ▶ La relation affine f doit
 - ▶ être la plus plate possible : ω est le plus petit possible.
 - ▶ avoir sa variable explicative dans un ϵ -tube autour de sa prédiction : $|y_i - f(x_i)| \leq \epsilon$.

Problème primal

$$\begin{cases} \arg \min_{\omega, b} \frac{1}{2} \|\omega\|_2^2 \\ |y_i - (\omega \cdot x_i + b)| \leq \epsilon \quad 1 \leq i \leq n \end{cases}$$

Relaxation des contraintes

Problème Il n'y a pas toujours de solution si ϵ est trop petit.

Relaxation des contraintes Les écarts $E_i \geq 0$ sont définis par

$$E_i = \begin{cases} 0 & \text{si } |y_i - (\omega \cdot x_i + b)| \leq \epsilon \\ |y_i - (\omega \cdot x_i + b)| - \epsilon & \text{sinon} \end{cases}$$

Réécriture du problème

$$\begin{cases} \arg \min_{\omega, b} \left\{ \frac{1}{2} \|\omega\|_2^2 + C \sum_{i=1}^n E_i \right\} \\ |y_i - (\omega \cdot x_i + b)| - \epsilon = E_i \quad 1 \leq i \leq n \end{cases}$$

Remarque : La fonction de coût est dite ϵ -insensitive.

Retour à un problème d'optimisation quadratique

Problème pas quadratique : Les contraintes ne sont pas linéaire en les variables ω , b et E_i à cause de la valeur absolue.

Variables d'écart Soient $\xi = (\xi_1, \dots, \xi_n)$ et $\xi^* = (\xi_1^*, \dots, \xi_n^*)$ des variables positives.

- ▶ Dans l'idée elles ont le même rôle que les variables d'écart à la marge.
- ▶
 - ▶ $\xi_i = 0, \xi_i^* = 0 \Rightarrow$ l'observation est dans l' ϵ -tube.
 - ▶ $\xi_i > 0, \xi_i^* = 0 \Rightarrow$ l'observation est au dessus de l' ϵ -tube.
 - ▶ $\xi_i = 0, \xi_i^* > 0 \Rightarrow$ l'observation est au dessous de l' ϵ -tube.
- ▶ Les vecteurs supports sont les observations en dehors de l' ϵ -tube.
- ▶ Il est nécessaire de définir deux nouveaux hyperparamètres :
 - ▶ ϵ : le rayon du tube dans lequel les observations doivent être pour ne pas pénaliser la régression.
 - ▶ C : le poids des variables d'écart.

Primal SVR

$$\left\{ \begin{array}{l} \arg \min_{\omega, b, \xi, \xi^*} \left\{ \frac{1}{2} \|\omega\|_2^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \right\} \\ y_i - (\omega \cdot x_i + b) \leq \epsilon + \xi_i \quad 1 \leq i \leq n \\ (\omega \cdot x_i + b) - y_i \leq \epsilon + \xi_i^* \quad 1 \leq i \leq n \end{array} \right.$$

Remarque : L'astuce du noyau peut aussi être appliquée dans ce cas :

$$f(x) = \sum_{i=1}^n (\lambda_i - \lambda_i^*) k(x_i, x) + b.$$

Table des matières

Données linéairement séparables

Classification et problème d'optimisation

Les marges souples

Données non linéairement séparables

Conclusion sur les classifieurs

Régression à vecteurs supports

Pour aller plus loin : Reformulation en problème d'optimisation

- Problème dual

- Les marges souples

- L'astuce du noyau

- La régression

Problème dual

Problème dual D'après les conditions Karush, Kuhn et Tucker, (ω^*, b^*) est solution optimal du problème primal si et seulement si $\lambda^* = (\lambda_1^*, \dots, \lambda_n^*)$ est solution du problème dual défini par

$$\left\{ \begin{array}{l} \arg \max_{\lambda} \left\{ \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j x_i \cdot x_j \right\} \\ \lambda_i \geq 0 \quad 1 \leq i \leq n \\ \sum_{i=1}^n \lambda_i y_i = 0 \end{array} \right.$$

Remarques :

- ▶ Le problème primal a $d + 1$ inconnues et N contraintes.
- ▶ Le problème dual a N inconnues et une contrainte en plus des contraintes de signes sur les λ_i .

Hyperplan optimal

Solution :

- ▶ La solution λ^* du problème dual permet de déduire ω^*

$$\omega^* = \sum_{i=1}^n \lambda_i^* y_i x_i.$$

- ▶ Les termes non nuls de cette somme sont ceux en lien avec les vecteurs supports.
- ▶ b^* se déduit des vecteurs supports et de $\omega^* \cdot x_i + b^* = y_i$
- ▶ L'hyperplan optimal est construit uniquement à partir des vecteurs supports.

Problème dual de la SVM à marge souple

Problème dual

$$\left\{ \begin{array}{l} \arg \max_{\lambda} \left\{ \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j x_i \cdot x_j \right\} \\ C \geq \lambda_i \geq 0 \quad 1 \leq i \leq n \\ \sum_{i=1}^n \lambda_i y_i = 0 \end{array} \right.$$

Les vecteurs supports :

- ▶ Soit $\xi_i = 0$ et le vecteur support est sur la frontière de la marge.
- ▶ Soit $0 < \xi_i < 1$ alors $y_i(\omega \cdot x_i + b) > 0$ donc le vecteur support est bien classée.
- ▶ Soit $\xi > 1$ alors $y_i(\omega \cdot x_i + b) < 0$ donc le vecteur support est mal classée.

Problème dual pour l'astuce du noyau

Problème dual



$$\left\{ \begin{array}{l} \arg \max_{\lambda} \left\{ \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j \phi(x_i) \cdot \phi(x_j) \right\} \\ C \geq \lambda_i \geq 0 \quad 1 \leq i \leq n \\ \sum_{i=1}^n \lambda_i y_i = 0 \end{array} \right.$$

► Pour des points t_1 et t_2 le noyau $k(t_1, t_2) = \phi(t_1) \cdot \phi(t_2)$

$$\left\{ \begin{array}{l} \arg \max_{\lambda} \left\{ \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j k(x_i, x_j) \right\} \\ C \geq \lambda_i \geq 0 \quad 1 \leq i \leq n \\ \sum_{i=1}^n \lambda_i y_i = 0 \end{array} \right.$$

Problème dual

Dual SVR

$$\left\{ \begin{array}{l} \arg \min_{\lambda, \lambda^*} \left\{ -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n (\lambda_i - \lambda_i^*)(\lambda_j - \lambda_j^*) x_i \cdot x_j \right. \\ \left. - \epsilon \sum_{i=1}^n (\lambda_i + \lambda_i^*) + \sum_{i=1}^n y_i (\lambda_i - \lambda_i^*) \right\} \\ C \geq \lambda_i \geq 0 \quad 1 \leq i \leq n \\ C \geq \lambda_i^* \geq 0 \quad 1 \leq i \leq n \\ \sum_{i=1}^n (\lambda_i - \lambda_i^*) = 0 \end{array} \right.$$

Solution du problème dual :



$$f(x) = \sum_{i=1}^n (\lambda_i - \lambda_i^*) x_i \cdot x + b.$$

► Seuls les vecteurs support contribuent dans ω et b .

Apprentissage statistiques pour l'IA

BUT SD 3A S6
2023-2024

Partie 3 : Réseaux de neurones

Christophe REYPE



© 2023 Christophe REYPE

Document sous licence Creative Commons Attribution – Partage dans les mêmes conditions 4.0 International (CC BY-SA 4.0)

<https://creativecommons.org/licenses/by-sa/4.0/>

Table des matières

Réseaux de neurones

- Perceptron et calculs logiques

- Réseau de neurones

- Le réseau de neurones en pratique

- Apprentissage des poids

Réseaux de neurones convolutifs

D'autres exemples de DeepLearning

Présentation du perceptron

Perceptron

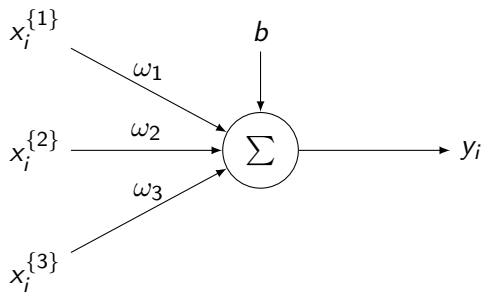
- ▶ Créé en 1950 par le psychologue américain Frank Rosenblatt.
- ▶ Inspiré de la première modélisation de neurone biologique de Walter Pitts et Warren McCulloch en 1943.
- ▶ Idée : adapté les paramètres du modèle de neurone en fonction d'exemples d'apprentissage.
- ▶ Classifieur biclasse.

Définition du perceptron

- ▶ Prend en entrée $x_i = (x_i^{\{1\}}, \dots, x_i^{\{d\}}) \in \mathbb{R}^d$,
- ▶ Utilise les paramètres $b \in \mathbb{R}$ et les poids $\omega = (\omega_1, \dots, \omega_d) \in \mathbb{R}^d$,
- ▶ Retourne en sortie $y_i = \text{signe}(b + \omega \cdot x_i)$:

$$y_i = \begin{cases} 1 & \text{si } b + \sum_{u=1}^d \omega_u x_i^{\{u\}} \geq 0 \\ -1 & \text{sinon.} \end{cases} \quad (1)$$

Représentation du perceptron



Remarques sur le perceptron

- ▶ Les paramètres du perceptron sont appris des données,
- ▶ L'algorithme est error-driven : les mises à jours visent à réduire les erreurs,
- ▶ La mise à jour des paramètres se fait par itération en parcourant toutes les observations,
- ▶ Une itération ayant parcouru toutes les observations est une époque (epoch).

Algorithme d'apprentissage du perceptron

Poser $b = 0$ et $\omega = 0$

- ▶ Tant que $b = 0$ et ω ne sont pas constants
 - ▶ Pour $i = 1, \dots, n$
 - ▶ $\hat{y}_i = \text{signe}(b + \omega \cdot x_i)$
 - ▶ si $\hat{y}_i \neq y_i$ poser $b = b + y_i$ et $\omega = \omega + y_i x_i$
- ▶ Renvoyer b et ω .

Remarques :

- ▶ Le perceptron peut être vu comme la descente de gradient stochastique minimisant le coût de l'erreur

$$\ell(\omega) = \sum_{i=1}^n \max \{0, -y_i \omega \cdot x_i - y_i b\}.$$

Cas linéairement séparable

Théorème : Si les observations sont linéairement séparables alors le perceptron aboutit en un temps fini aux paramètres d'un hyperplan séparateur.

Cas linéairement séparable :

- ▶ Le nombre d'itération pour la convergence est indépendant de d et n . Il ne dépend que de la marge $\gamma = \min_i \{y_i \omega_i \cdot x_i\}$: plus γ est grand et plus la convergence est rapide.
- ▶ La vitesse de convergence **dépend** de n .

Généralisation du perceptron à K classes

One VS Rest : Construire K perceptrons chacun séparant une classe avec les autres.

Perceptron multiclasse : Entraîner un perceptron avec des paramètres spécifiques à chaque classe. La classe associée à l'individu i est celle qui maximise $b + \omega \cdot x_i$.

Les mains dans le cambouis

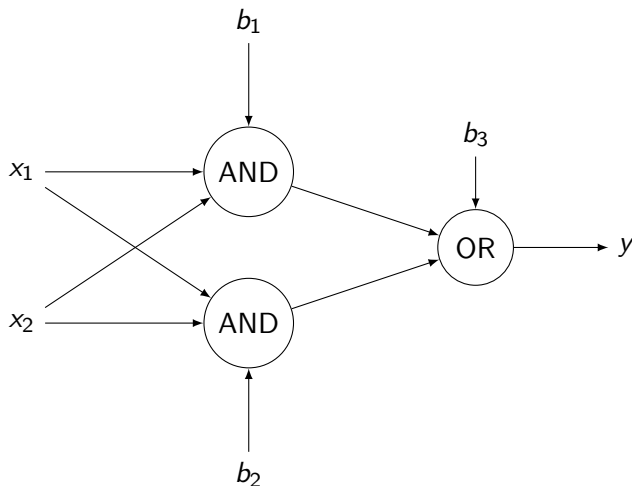
- ▶ Construire la table des vérités pour les opérateurs logiques AND, OR, XOR.
- ▶ Représenter géométriquement les tables de vérités précédentes.

Hiver de l'IA

- ▶ Le perceptron capable de résoudre TOUS les problèmes de classification linéairement séparable.
- ▶ Le perceptron NE PEUT PAS résoudre des problèmes de classification non linéairement séparable $\Rightarrow$ premier "hiver de l'IA".

Résolution des opérations logiques

Décomposition de l'opération : $x_1 \text{ XOR } x_2 = (x_1 \text{ AND NOT}(x_2)) \text{ OR } (\text{NOT}(x_1) \text{ AND } x_2)$



Perceptron multicouche

Théorème : Toute fonction de n booléens à valeurs booléennes peut être calculée par un réseau de perceptrons à une couche cachée.

Construction du perceptron multicouche (multilayer perceptron MLP) :

- ▶ Couche d'entrée : d neurones pour représenter $x_i = (x_i^{\{1\}}, \dots, x_i^{\{d\}}) \in \mathbb{R}^d$,
- ▶ Couches cachées : chaque neurone d'une couche cachée est lié à tous les neurones de la couche précédente,
- ▶ Couche de sortie : une couche de neurones dont chaque neurone est lié à tous les neurones de la dernière couche cachée.

Les mains dans le cambouis

- ▶ Dessiner le MLP ayant 4 neurones sur la couche d'entrée, 2 couches cachées la première ayant 3 neurones et la seconde ayant 2 neurones et enfin une couche de sortie ayant 3 neurones.
- ▶ Combien de paramètres sont nécessaires pour définir le MLP précédent ?

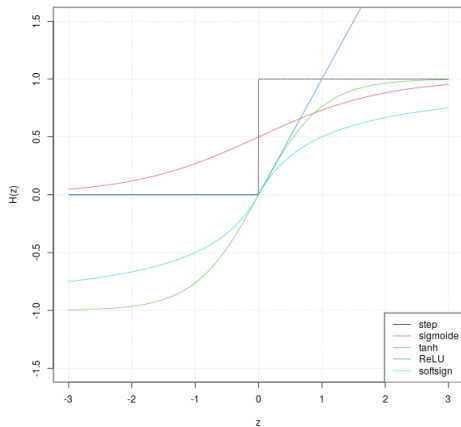
Cas non linéaire

Limitation : Le réseau de neurones ne peut traiter que des cas où la réponse attendue est une combinaison linéaire des entrées.

Généralisation

- ▶ Une fonction dite fonction d'activation peut être appliquée à la sortie d'un neurone.
- ▶ Une fonction d'activation est choisie par couche.
- ▶ Soit H une fonction d'activation et $a_{\ell,j}$ la sortie du neurone j de la couche ℓ , la sortie est transformée en $H(a_{\ell,j})$.
- ▶ Exemples de fonction d'activation :
 - ▶ step : $s(a) = \mathbb{1}_{a \geq 0}$
 - ▶ sigmoïde : $\sigma(a) = \frac{1}{1+e^{-1}}$
 - ▶ tangente hyperbolique : $\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}} = 2\sigma(2a) - 1$
 - ▶ Rectified Linear Unit : $\text{ReLU}(a) = \max\{0, a\}$
 - ▶ softsign : $\text{softsign}(a) = \frac{a}{1+|a|}$

Visualisation de fonctions d'activation



Théorème d'approximation universelle

Théorème d'approximation universelle Soient ϕ une fonction strictement croissante continue bornée et C un compact de $\mathbb{R}^d$. Pour tout $\epsilon > 0$ et f continue sur C , il existe $M \in \mathbb{N}$ et pour tout $m \in \{1, \dots, M\}$, $v_m, b_m \in \mathbb{R}$, $\omega_i \in \mathbb{R}^d$ tels que la fonction

$$F(x) = \sum_{m=1}^M v_m \phi(\omega_m \cdot x + b_m)$$

vérifie

$$\forall x \in C, |F(x) - f(x)| \leq \epsilon$$

Réseau de neurones pour la classification biclasse

Classification biclasse L'objectif est à partir de $x \in \mathbb{R}^d$ de prédire la classe de $y \in \{0, 1\}$. Le réseau de neurones est composé de

- ▶ une couche d'entrée à d neurones
- ▶ des couches cachées
- ▶ une couche de sortie à 1 neurone avec une fonction d'activation qui émet une sortie entre 0 et 1.

Réseau de neurones pour la classification multiclasse

Classification multiclasse L'objectif est à partir de $x \in \mathbb{R}^d$ de prédire la classe de $y \in \{0, 1\}^L$. Le réseau de neurones est composé de

- ▶ une couche d'entrée à d neurones
- ▶ des couches cachées
- ▶ une couche de sortie à L neurones. Pour chacune des sorties, la formule de "SoftMax" suivante est appliquée

$$y_l = \frac{\exp(a_l)}{\sum_{l=1}^L \exp(a_l)}$$

où a_l est la sortie du neurone l de la couche de sortie et la classe prédite est celle dont y_l est le plus élevé.

Réseau de neurones pour la régression

Régression L'objectif est à partir de $x \in \mathbb{R}^d$ la prédiction $y \in \mathbb{R}^D$.

Le réseau de neurones est composé de

- ▶ une couche d'entrée à d neurones
- ▶ des couches cachées
- ▶ une couche de sortie à d neurones sans fonction d'activation.

Apprentissage des poids

- ▶ Le choix des poids se fait par l'apprentissage des poids en fonction des données d'entraînement.
- ▶ L'apprentissage se fait par minimisation d'un coût d'erreur global appelé fonction de perte.
- ▶ Cette minimisation se fait par descente de gradient.
- ▶ La descente de gradient consiste à chercher le minimum d'une fonction à partir d'un point de départ vers le minimum de manière itérative au moyen des dérivées partielles de la fonction.
- ▶ Les poids optimaux sont obtenues par la méthode de rétropropagation du gradient (backpropagation).

Fonction de perte

Le coût d'erreur moyen ou risque empirique est défini par

$$E(\omega) = \sum_{i=1}^n E_i(\omega)$$

Les erreurs classiques E_i sont

- ▶ classification biclasse :

$$E_i(\omega) = -y_i \log(\hat{y}_i) - (1 - y_i) \log(1 - \hat{y}_i)$$

- ▶ classification multiclasse : $E_i(\omega) = \sum_{l=1}^L y_{i,l} \log(\hat{y}_{i,l})$

- ▶ régression : $E_i(\omega) = \frac{1}{2} \|y_i - \hat{y}_i\|_2^2$

avec par convention $0 \log(0) = 0$.

Descente de gradient

Principe :

- ▶ En 1-D, une fonction atteint un optimum (maximum ou minimum) uniquement lorsque sa dérivée s'annule.
- ▶ En d -dimensions, une fonction atteint un optimum uniquement si son gradient (vecteur contenant ses dérivées partielles) est le vecteur nul.
- ▶ En notant $\eta > 0$ le taux d'apprentissage ou pas de descente :

$$\omega \leftarrow \omega - \eta \nabla E$$

Rétropropagation du gradient

Principe :

- ▶ La mise à jour des paramètres se fait par la descente du gradient.
- ▶ La mise à jour des paramètres commence par la couche de sortie et se propage vers la couche d'entrée : rétropropagation.
- ▶ La mise à jour du poids liant la le neurone j dernière couche cachée au neurone u de la couche de sortie S est :

$$\omega_{u,j} \leftarrow \omega_{u,j} - \eta \frac{\partial E}{\partial \omega_{u,j}}(\omega).$$

- ▶ Le terme $\frac{\partial E}{\partial \omega_{u,j}}(\omega)$ est exprimé en fonction des sorties a_i de la couche précédente :

$$\frac{\partial E}{\partial \omega_{u,j}}(\omega) = \frac{\partial E}{\partial a_i} \frac{\partial a_i}{\partial \omega_{u,j}}$$

Conclusion

Limites :

- ▶ Grand nombre de paramètres,
- ▶ Besoin de beaucoup de données d'entraînement,
- ▶ Temps de calcul important pour l'estimation des paramètres,
- ▶ Risque de sur-apprentissage,
- ▶ Pas interprétable : boîte noire.

Eviter le sur-apprentissage

- ▶ arrêt prématuré de l'entraînement (early stopping) : soit après un nombre fixé *a priori* d'époque soit quand l'erreur sur les données de validation augmente
- ▶ régularisation : la fonction à minimiser n'est plus l'erreur empirique mais $E(\omega) + \lambda \|\omega\|_2^2$ avec $\lambda > 0$ un hyperparamètre.

Table des matières

Réseaux de neurones

Réseaux de neurones convolutifs

- Réapparition des réseaux de neurones

- Convolution

- Banc de filtres

- Pooling

D'autres exemples de DeepLearning

Réapparition des réseaux de neurones

A partir du concours de reconnaissance d'images ImageNet de 2012, c'est le retour en force des réseaux de neurones.

Le retour des réseaux de neurones est rendu possible par

- ▶ le développement des outils informatiques
- ▶ la quantité importante de données
- ▶ le développement des bibliothèques logicielles
- ▶ la nouvelle forme des réseaux de neurones

Passage à l'apprentissage profond (deep learning)

Le réseau profond (deep neural network) est un MLP avec plus de couches cachées d'où la notion de profondeur.

Exemple : Concours ImageNet 2012

<https://image-net.org/challenges/LSVRC/2012/> Le réseau de l'équipe SuperVision est composé de 60 millions de paramètres et 650000 neurones.

Convolution

- ▶ La couche de convolution est une couche de neurones.
- ▶ Une couche de convolution est utilisé lorsque l'information d'un neurone est liée à celle d'autres neurones (ex : analyse d'image).
- ▶ La convolution découle de la notion de filtre.

Traitement du signal

L'utilisation de fonction d'activation pour transformer un signal entrant dans un neurone donne l'idée d'appliquer des filtres à ces signaux.

Filtre Un filtre est une fonction H transformant le signal d'entrée a en un signal de sortie s et possédant les propriétés suivantes

- ▶ Linéarité : soit deux signaux a_1 et a_2 et $\alpha \in \mathbb{R}$ alors
$$H(a_1 + \alpha a_2) = H(a_1) + \alpha H(a_2)$$
- ▶ Continuité
- ▶ Invariance par translation : si $\forall t, H(a(t)) = s(t)$ alors
$$\forall t, \tau, H(a(t - \tau)) = s(t - \tau)$$

Sens des propriétés d'un filtre

- ▶ Linéarité : la sortie correspondant à la superposition de deux signaux est la superposition quand les signaux sont considérés comme indépendants.
- ▶ Continuité : Une petite perturbation du signal d'entrée est traduite par une petite perturbation du signal de sortie.
- ▶ Invariance par translation : Le traitement du signal sera le même qu'importe sa position. Par exemple le traitement d'un objet sera le même qu'il soit au milieu ou non de l'image.

Convolution

Pour tout filtre H , il existe une fonction h tel que pour tout signal d'entrée a et sa sortie s

$$H(a) = h * s$$

où $*$ est le produit de convolution défini par

$$\forall t \in \mathbb{R}, h * s(t) = \int_{\mathbb{R}} h(t - u)s(u)du = \int_{\mathbb{R}} h(u)s(t - u)du.$$

La fonction h est appelée noyau de convolution.

La convolution pour le traitement d'un signal

En pratique, les signaux numériques sont discrets. Le produit de convolution s'écrit alors

$$\forall n \in \mathbb{Z}, h * s(n) = \sum_{k \in \mathbb{Z}} h(k)s(n - k).$$

En général h est non nul pour un nombre fini de valeur.

Cas de l'analyse d'image

- La convolution s'écrit

$$\forall (m, n) \in \mathbb{Z}^2, h * s(m, n) = \sum_{(k, l) \in \mathbb{Z}^2} h(k, l)s(m - k, n - l).$$

- Les neurones d'entrées lisent le signal.
- La couche cachée calcule les produit de convolution. Il y a autant de neurones dans cette couche que dans la couche d'entrée. Ces neurones sont liés uniquement au neurones intervenant dans le calcul de la convolution. Le poids de ces neurones est le coefficient du noyau.

Exemple de convolution

Exemple : Soit un signal provenant de n neurones d'entrée et un noyau dont la taille de support est 3. Le produit de convolution est :

$$\forall i \in \{1, \dots, n\}, h * s(i) = \sum_{k \in [-1, 0, 1]} h(k)s(i - k).$$

Le neurone numéro i de la couche cachée est reliée aux neurones $i - 1$, i et $i + 1$ de la couche d'entrée avec pour poids respectivement $h(1)$, $h(0)$ et $h(-1)$.

Remarques :

- ▶ Le poids des neurones ne dépend pas du nombre de neurones.
- ▶ Il faut traiter différemment les bords ($i = 1$ et $i = n$).

Exemples de noyau de convolution

Des exemples de filtres pour le traitement d'images sont disponibles sur [https://fr.wikipedia.org/wiki/Noyau_\(traitement_d'image\)](https://fr.wikipedia.org/wiki/Noyau_(traitement_d'image))

Banc de filtres

Un banc de filtres est l'application sur un même signal de filtres autonomes.

- ▶ Chaque filtre sera calculé au moyen d'une couche cachée différente indépendantes utilisées sur la même entrée.
- ▶ Pour une image en couleur, chaque pixel est composé de 3 composantes : rouge, vert, bleu. Il faut ici 3 noyau de convolution donc 3 couches cachées.
- ▶ Pour un banc de filtres composé de d filtres, il faut une couche d'entrée d'épaisseur d : l'entrée est répétée d fois pour chaque couche cachée.
- ▶ Il est possible d'appliquer une fonction d'activation sur chaque couche convolutive.

Pooling

Le nombre de paramètres dans une couche convolutif dépend de la taille du support.

- ▶ En général pour les images, les noyaux sont de taille 3×3 ou 5×5 et si il y a trois couleurs $3 \times 3 \times 3$ ou $5 \times 5 \times 3$
- ▶ Pour intégrer de l'information plus lointaine dans l'image, il y a deux stratégies
 1. Faire des convolutions successives
 2. Faire du pooling : réduire la taille de l'image en la découpant en sous-ensembles. Le signal devient alors le signal obtenu sur ces sous-ensembles. Le signal d'un sous-ensemble est obtenu par la fonction de pooling.

Max pooling

Un exemple de pooling : le max pooling. Dans ce cas la fonction de pooling est le maximum.

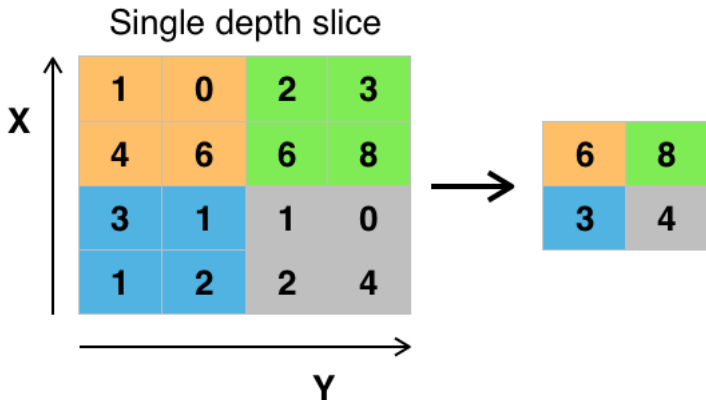


Figure – Aphex34, CC BY-SA 4.0

<https://creativecommons.org/licenses/by-sa/4.0>, via Wikimedia Commons

Schéma explicatif de AlexNet

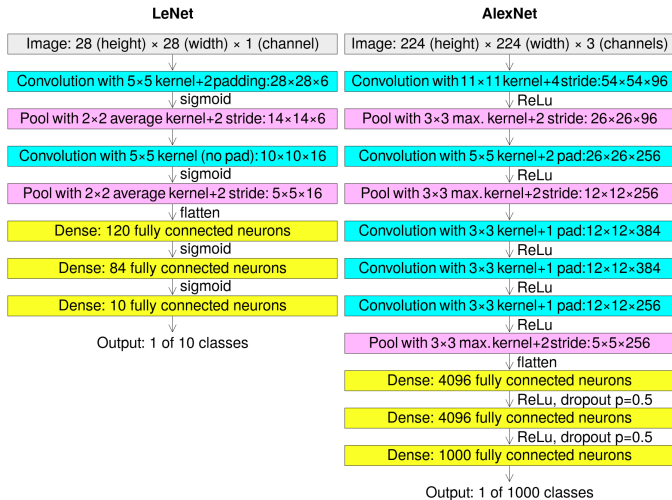


Figure – Cmglee, CC BY-SA 4.0

<https://creativecommons.org/licenses/by-sa/4.0>, via Wikimedia Commons

Sur-apprentissage

Problème : Le risque de sur-apprentissage est plus important qu'un MLP classique.

Dropout : Pour éviter le sur-apprentissage, lors de la phase d'apprentissage, chaque neurone d'une couche proche de la sortie n'est pas mis à jour avec une probabilité p .

Table des matières

Réseaux de neurones

Réseaux de neurones convolutifs

D'autres exemples de DeepLearning

Réseau de neurones récurrents

Natural Language Processing

Réseau de neurones récurrents

Les réseaux de neurones récurrents (Recurrent Neural Network) permettent de garder en mémoire des informations comme du contexte grammatical pour un traducteur ou le début d'un mot dans la reconnaissance vocale.

Particularités :

- ▶ La sortie dépend de l'entrée mais aussi de ce qu'il y a stocké en mémoire.
- ▶ Le réseau Long Short Term Memory est utilisé pour augmenter la capacité de la mémoire : pour chaque moment le réseau décide de l'information à conserver et de l'information à ajouter au registre. La sortie est donc le résultat de l'entrée et du registre.

Natural Language Processing

Le NLP est utilisé pour la traduction.

Le NLP repose sur les techniques suivantes :

- ▶ words embeddings : les mots sont représentés dans un espace. Des mots proches ont des sens similaires.
- ▶ architecture encodeur-décodeur : la phrase est traitée par un LSTM appelé encodeur. Le décodeur convertie le résultat de l'encodeur en une suite de mots.
- ▶ apprentissage par transfert : c'est le fait de réutiliser tout ou une partie d'un modèle déjà entraînée pour construire un nouveau modèle.
- ▶ mécanisme d'attention : c'est le fait de traiter une information avec différents niveau d'acuité